Top-down Parsing

CS 1622

Jarrett Billingsley

Class Announcements

- how was your weekeeeeend
- project 2 will come out this weekend so you'll have 2 weeks for it

Parsing

A generative view

- one way to use a grammar is to **produce** sentences in the language.

```
Exp:    Id | Num | Parens
Parens: '(' Exp+ ')'
Id:     <id token>
Num:    <int literal token>
```

we can produce valid sentences by starting at the **root rule** and repeatedly replacing nonterminals with their right-hand sides.

```
   Exp
=> Parens
=> '(' Exp+ ')'
=> '(' Exp Exp Exp ')'
=> '(' Id Id Num ')'
=> '(' 'add' Id Num ')'
=> '(' 'add' 'x' Num ')'
=> '(' 'add' 'x' '3' ')'
```

this sequence is called a **derivation.**

now suppose we were given `(add x 3)` as our *input*. **could we work *backwards* to figure out how to get to Exp?**

That's what parsing is.

- given a **grammar** and an **input string...**
- **parsing** is figuring out **the derivation that was needed** to produce the input string. (and if there *is* no such derivation, it's an error!)
- we represent the derivation as... an **abstract syntax tree!**

```
Exp: Id | Num | Parens
Parens: '(' Exp+ ')'
Id:     <id token>
Num:    <int token>
```

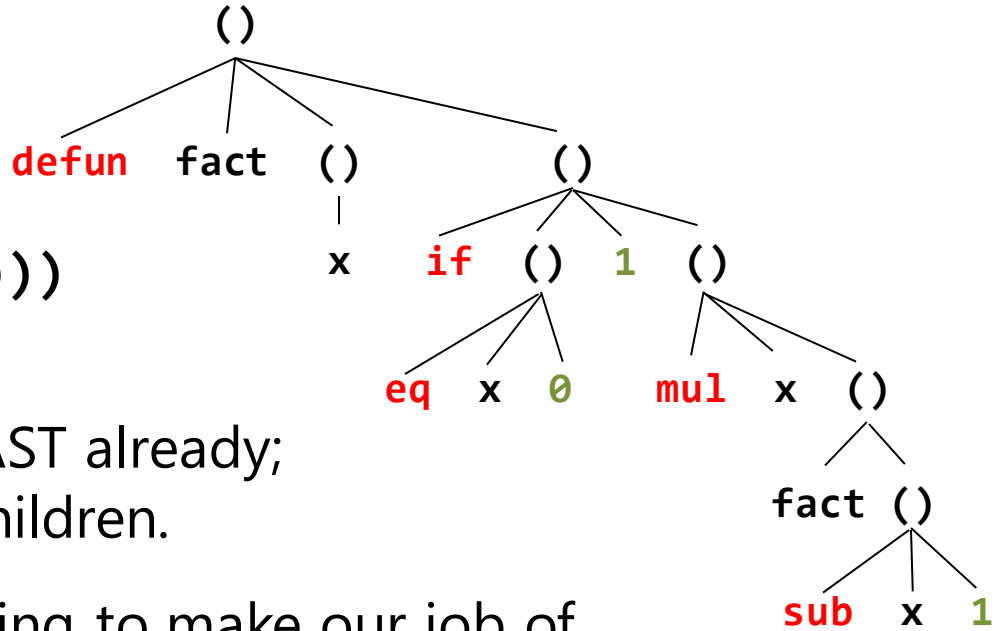
Input	Output AST
`e`	`Id("e")`
`(e)`	`Parens(Id("e"))`
`(1 2)`	`Parens(Num(1), Num(2))`
`(e`	`error!`
`e)`	`error!`
`()`	`error!`

the last three would benefit from some *error messages* that help the programmer.

Okay what *is* this language?

- this is a variety of **Lisp,** a language family that started in 1958.
- we're not going to get into what it *means,* but it's a very easy-to-parse example language.

```
(defun fact (x)
  (if (eq x 0)
    1
    (mul x (fact (sub x 1)))))
```



Lisp code is basically an AST already; parentheses group children.

this is going to make our job of parsing much easier.

So how do we do it?

- well it's a bit like lexing:
  - we start at the beginning, looking at *tokens* one-by-one.
  - based on the token, we decide which **rule** we should use.
  - if none of the rules apply, or if we see something that we don't expect, we can give an **error.**
- speaking of which...

Error Handling in Rust

time check: ≤ 17

The Result type

- Rust doesn't have exceptions like Java does.
- if your function needs to indicate failure, it returns a **Result.**

```
Result<T, E>
```

T is the type of value returned on **success.**

E is the type of value returned for **errors.**

if the function doesn't return any value on success, you can use `()` – this is Rust's `void`.

any type can be used for errors, but typically it's an enum.

```
Result<Box<AstNode>, ParseError>
```

this is a bit unwieldy, so we can use a type alias to shorten it:

```
type ParseResult = Result<Box<AstNode>, ParseError>;
```

Err, ok...

- you create Results with the **`Ok()`** and **`Err()`** constructors.

```
if thing.is_bad() {
    // common to use 'return' to immediately
    // leave the function. kinda like throwing
    return Err(ParseError::whatever());
}
```

you can **match** on a Result using **`Ok()`** and **`Err()`**:

```
match parse_thing(input) {
    Ok(ast) => { println!("{:?}", ast); }
    Err(e)  => { println!("o no: {}", e); }
}
```

match patterns **declare local variables** usable in their code blocks, if that was unclear before.

???????

- when using Result-returning functions, a really ugly pattern appears:

```
match step_one(input) {
  Ok(a) => {
    match step_two(a) {
      Ok(b) => {
        match step_three(b) {
          Ok(c) => {
            return Ok(c.to_string());
          }
          Err(e) => { return Err(e); }
        }
      }
      Err(e) => { return Err(e); }
    }
  }
  Err(e) => { return Err(e); }
}
```

this is terrible. instead:

```
let a = step_one(input)?;
let b = step_two(a)?;
let c = step_three(b)?;
Ok(c.to_string())
```

x? means, "if **x** is an error, return it; otherwise, give me the success value.

we'll be seeing this a lot in parsing code!

Recursive Descent

time check: ≤ 34

Can we intuit our way through this?

- with this grammar, and this sequence of tokens as our input…

```
Exp:    Id | Num | Parens
Parens: '(' Exp+ ')'
Id:     <id token>
Num:    <int literal token>
```

```
'(', '(', 'hi', '5', ')', ')'
```



we are here. our ultimate goal is to build an **Exp.** this token is a left-paren, so **which rule** do we think applies here?

Parens!

after the left-paren, we should see **one or more Exps,** followed by a right-paren.

but we're *already* trying to parse an **Exp**. how do we parse a new **Exp** without finishing this one?

The rules are recursive, so…

- recall that all CFGs have this **recursive rule structure.**
- well if the dependencies between the rules are recursive…
  - why not use recursive *functions* to model the rules?

```
Exp: Id | Num | Parens
Parens: '(' Exp+ ')'
Id:     <id token>
Num:    <int token>
```

let's write some **pseudocode** to get our thoughts down.

```
fn parse_exp() {
  match self.cur() {
    Id     => parse_id(),
    IntLit => parse_num(),
    LParen => parse_parens(),
  }
}
```

```
fn parse_parens() {
  eat_an_lparen();
  let exps = …parse_exp()…
  eat_an_rparen();
  return Parens(exps);
}
```

(looping happens here)

Believe it or not, that's pretty much right!

- what we've just (pseudo-)written is a **recursive descent parser:**
  - **recursive,** because uh, it is!
  - and **descent,** because we start at the **root rule,** and **descend** into the other rules until we get to the terminals.
- the issue with "trying to parse an Exp while parsing an Exp" is solved **by using recursion**
  - when you recurse, the caller rule's execution is paused, and it **remembers its place** in the rule.
  - it can then resume parsing after the callee rule has done its work.
- so how can we be a little less "pseudo" about this code?

From Rules to Rust

- each grammar rule is really a **list of steps,** and translating them to *real* code is relatively straightforward:

```
Parens: '(' Exp+ ')'
```

this says that to parse a **`Parens`**:

1. expect a `'('` token and skip it.
2. parse one **`Exp`**, since that's the minimum number.
3. while we don't see a `')'` token,
   - keep parsing **`Exp`**s and put them into a list.
4. expect a `')'` token and skip it.

if all of these steps succeed, we can create an **`Parens`** AST node and return it!

Other metalanguage rule correspondences

- there's a nice correspondence between the things we see in the grammar metalanguage and the patterns we use in our code.

A B	**sequencing:** parse an **A**; if that succeeds, then parse a **B**.
A | B	**alternation:** an `if-else` (or a `match`); either parse an **A** *or* parse a **B**.
A*	**0+ repetition:** `while` the next token looks like an **A**, keep parsing **A**s.
A+	**1+ repetition:** parse an **A**, then do a `while` loop like for 0+.
A?	**optional:** `if` the next token looks like an **A**, parse it.

From rules to AST nodes

- similarly, when we design our AST nodes, these metalanguage rules imply different data structures:

A B	**a struct,** with an A field and a B field.
A | B	**an enum,** where A and B are two variants.
A* **A+**	**an array/vector,** whose length is the number of repetitions.
A?	**an `Option<A>` field,** since it might not exist.

The example

- now let's have a look at the new **`parsing_lisp`** example I added.
  - **`src/lib.rs`** is where all the goodies are.
  - the **`Token`** and **`AstNode`** types are familiar to you by now.
    - there's an extra **`Program`** rule in the grammar though.
  - **`ParseError`** is an example of an error enum.
  - **`Parser`** looks a lot like **`Lexer`** from your project...
    - except it iterates over **`Token`**s, not **`char`**s.
  - and there are three parsing methods for the three main rules.
- this real parser handles errors, too.
  - the **`expect_blah`** methods and the **`match`** in **`parse_exp`** deal with unexpected terminals (tokens) in various positions.
    - they also give customized errors for each of these possibilities.
    - there's no location info, because it's an example. :B

Limits of Recursive Descent

time check: ≤ 68

If it works so well, why not use it for everything?

- there are lots of cases where recursive descent works great!
  - `import java.util.Arrays;`
  - `use std::blah;`
  - `fn name() { … }`
  - `class A { … }`
  - `if x == y { … }`
  - `match value { … }`
- but there's something all of these things have in common:
  - **they all *start* with a token that *unambiguously* indicates which rule should be used to parse them.**
  - if you don't have that, recursive descent gets a lot harder to use.
- but, what kind of code looks like that?

Expressions!

- we want to be able to write expressions like we do in math.
  - that is, we want `4 + 5`, not `(add 4 5)`.
- but this presents a few problems.

Input	Output	
`x + y + z`	`(x + y) + z` *or* `x + (y + z)` ?	either is fine..? what if it were *, -, or / ?
`x + 4 * y`	`(x + 4) * y` *or* `x + (4 * y)` ?	the second one is right.

and if we allow ** for exponents, like Python:

`2 ** x ** 2`	`(2 ** x) ** 2` *or* `2 ** (x ** 2)` ?	the second one is right.

Even worse...

- **how do we know what kind of expression we're looking at?**

```
(3 + x + y * 2) / 50
```


let's say our parser is looking at this token....



...how will it know it's a division until it gets here?

so what do we do? **look ahead?** how many tokens? 5? 10? *is there a limit?*

what we would have to do here is **try each *possible* parse,** and **backtrack if we mess up.**

OH NO, BACKTRACKING

- this is an **exponential time ($O(2^n)$) algorithm!** just awful!
  - and from an intuitive sense, it feels kind of silly to commit to parsing something when you *don't actually have enough information yet* to parse it.
- so for these (and other similar) situations, there is another way to parse that's a little more mind-bending, but more powerful.
  - that's for next time!

Error Reporting

time check: ≤ 85

Compilers have a bit of a reputation

- how many times have you gotten compiler errors that:
  - seemed totally confused by very simple/common typos, like missing a semicolon or comma or closing brace?
  - pointed 100 lines after where the actual error was?
  - used weird terminology, like "specifier-qualifier-list"?
  - gave almost no information, like "syntax error"?
- is it because the compiler writers suck?
  - no
  - ***good error reporting is really hard to do right, okay,***

Who are error messages for?

- I've said it many times: programming languages exist for **humans.**
- so when a compiler gives errors, they should be… for **humans!**
- but there are two things working against us here.

the compiler is a dumb algorithm that **doesn't understand our human mistakes**

```
f(3 4)
----^
expected `)' or `,' in
argument-list, not `4'
```

people who write compilers tend to forget that *other* people **don't know how they work**

```
(substitution of deduced
template arguments resulted
in errors seen above)
```

Carry on, my wayward son

- you know how you sometimes get like 1000 error messages?
  - and it's all because you forgot ONE closing brace?
- this is **error recovery:** instead of stopping at the first error, the compiler *tries* to keep going.

```
enum E {
  X,
  Y
    // uh oh.

fn func() {
  let x = E::X // oops
  (blah).y();
}
```

but how would you *algorithmically* determine where the closing brace should go?

this could even *parse* differently depending on where we infer the semicolon "should" be.

To recover or not?

- in the past, languages were simpler, and so was error recovery.
  - compilers also ran on huge computers in batch jobs.
  - since a compile might take hours, it was useful to report as many errors as possible.
- but now, a really, *really* common thing to do is:
  - compile, fix the **first error,** repeat
    - (I really recommend you do this if you don't already)
- so maybe stopping at the first error is fine?
  - but what about IDEs?
  - they can show multiple errors inline with the code
  - so maybe there's still some use to it?
    - or does it just make using an IDE more annoying? :^)
- personally I think lexing/parsing error recovery is pointless, but giving multiple *semantic* errors can still be useful. sometimes.

A philosophy for good errors

- there are three ways to improve error messages.

1. say **where it is**

```
test.foo(9:17)
```

bare minimum...

```
test.foo(9:17):
  for num in 10 {
                ^ here
```

better!

2. give a **unique message** for that particular error

```
expected ',', not '{'
```

ehh...

```
for-loop missing upper bound
```

better!

3. extra credit: **tell the user how they might fix it**

```
help: maybe you meant "for num in 10, something"?
                      ^^^^^^^^^^^^
```

First: location information

- the **lexer** can associate a **line and column** with each token.
  - well, some tokens span more than one column…
  - or even more than one line, if you allow multi-line strings!
  - so maybe each token should have a *range* of locations? hmm
- when parsing, that info can be carried from the tokens **into the AST.**
  - that way, we can report errors during semantic analysis.
  - or maybe we want to **refer back to the token list** somehow…?
    - there are lots of ways to implement this!

Second: accurate error messages

- it's really easy to make unhelpful, uninformative messages, sadly!
- **giving accurate errors** is largely about customizing the error messages for each place an error could happen.
  - looking through the parser code, every **?** is a potential error spot.
  - in **`parse_paren_exp`**, there are several of these.
- **matches** are another place ripe for better errors:
  - in **`parse_exp`,** there are a number of possible options, so we should **list the options** in the error message, like "expected identifier, integer, or parenthesized expression, not blah blah"
- one possibility is to pass some kind of "error context" to the parsing functions which can be used to tailor the messages better
  - since **`parse_exp`** is called from multiple places, the error message might be different depending on who calls it.

Third: giving help

- this is extremely broad and it's up to you what to say.
  - it's really kind of an extension of the previous rule.
- have a look at the error messages Rust gives.
  - sometimes it's as simple as "remove this semicolon."
  - sometimes it links to the appropriate part of the docs.
  - sometimes it even detects common mistakes and explains why they won't work. it's amazing!